

OTL Platform

Master Brief

Replacing PushPress with our own CRM, member web app and iOS app
Complete research record, findings and implementation plan

CrossFit OTL

Compiled 19 September 2026

What this contains

Part I reproduces every message from the originating conversation **verbatim and unedited**. Part II records every research finding with its source and how it was verified. Part III is the plan: migration, architecture, staff mode and POS, CAP ingestion, infrastructure. Parts IV–VI are the action checklist, open questions and source URLs. Appendices cover the extraction tooling and the vault file map.

Endpoint inventories were read from official SDK source code rather than documentation. DNS findings come from live public queries. Apple's guidelines are quoted verbatim.

Contents

How to use this document	4
Part I — Conversation transcript (verbatim, unedited)	5
Message 1 — 2026-09-19	5
Message 2 — 2026-09-19	5
Message 3 — 2026-09-19	5
Message 4 — 2026-09-19	6
Message 5 — 2026-09-19	7
Part II — What was researched and found	9
1. PushPress Platform API — what exists and what doesn't	9
2. The CAP -> Train credential — whose it is	10
3. SugarWOD — the control case	11
4. Stripe — the best news in the project	11
5. Apple App Store — quoted verbatim	12
6. Instagram — half the feature is achievable	13
7. crossfit-otl.com — live DNS audit, 2026-09-19	13
8. PushPress export & reporting routes — better than expected	13
9. Stripe Terminal / Tap to Pay on iPhone — POS requirements	14
Part III — The plan	15
A. Migration	15
B. Architecture	17
C. One app, role-gated — staff mode and POS	18
D. CAP programming into a fully independent CRM	20
E. Infrastructure — domain, DNS, email	21
F. Scope — the honest number	21
Part IV — Action checklist	23
This week, independent of everything else	23
At the Mac mini	23
Then, to build	23

Part V — Open questions blocking decisions	24
Part VI — Sources	25
Appendix A — pushpress_extract.py	27
Appendix B — Vault file map	28
Related	28

What this is

The complete record of the OTL platform project as of **2026-09-19**: every message in the originating conversation reproduced **verbatim and unedited**, every research finding with its source, and the full plan that came out of it. Written so the Mac mini has one file to open.

A PDF of this document sits beside it as `OTL Platform – Master Brief.pdf`.

The three headline findings, if you read nothing else

Members almost certainly never re-enter a card. PushPress connects the gym's *own* Stripe account, so the new CRM just points at it. **Workout results, benchmarks and PRs have no export path anywhere in this industry.** Extract them while still a paying PushPress customer — access dies with the account. ****crossfit-otl.com** has no SPF, DKIM or DMARC.** CRM email sent from it today goes to spam, and the domain is spoofable right now.

How to use this document

Part	Contents
I	Conversation transcript — verbatim, unedited
II	Research findings, with sources and verification method
III	The plan — migration, build, POS, CAP ingestion, infrastructure
IV	Action checklist
V	Open questions blocking decisions
VI	Source URLs
App. A	<code>pushpress_extract.py</code> — full source
App. B	Vault file map

Part I — Conversation transcript (verbatim, unedited)

Reproduced exactly as sent

Transcribed speech, so spelling and punctuation are as dictated — including Paul for pull, hotel for OTL, Jim for gym, abs for apps, and I flee to us for MyFleetOS. Left untouched deliberately: this is the source record.

Message 1 — 2026-09-19

it figured out. So we're trying to build our um, gym software app, and basically we use a software called PushPress. It manages all of our contacts and payments and engagement, member engagement, KPIs, all of that stuff. But it also has our workout software that automatically pulls in from the CrossFit affiliate toolkit. We need to find that API key and determine how to create our own so that all of the workouts are automatically synced and then we can use the the push press software to track all of our members' uh, workout scores and PRs and all of the above.

Message 2 — 2026-09-19

No, the API key would have to be pulled or done somehow by the train app and the train app get Paul workouts from Crossfit affiliate programming

Message 3 — 2026-09-19

I also need to build an app that's connected to the CRM that we are going to build to replace push press when I get back to my Mac mini. I will give you the logins in a chrome browser for push press and any other system that you'll need connections to as well as OTL app and I'll have you scrape the whole system because what we need to do is we need to take the push press and we need to make sure that we can transfer everybody over to our new system. Need to transfer their information for cards, bank accounts committee club streak check-in, benchmarks, workouts, profile pictures everything you could possibly imagine from push press to our own CRM as well as old members new leads in any tags that are associated with every single member that way when we transfer them over, it's not like we're asking them to set up a new bank account set up a new card either email at the emergency contact we can transfer all of that over push press, but we currently use is using the payment processor Stripe so we already have a bunch of Stripe information set up for all of the different apps that we already have like Classic Pro I flee to us Longhorn Forge holdings all of the above have Stripe so we could just simply trade or move the CrossFit OTL Stripe account to be hosted on our new CRM so that we can move the payments over to the new CRM without having to go through a big headache of potentially add asking everybody to add a payment method and then the goal is I'll give you the OTL app on iPhone mirroring on my Mac mini for you to scrape that OTL app is not our own individual app where we have control over it and hosting it. It is a push press hosted and created app so we won't be able to carry that app over meaning when we build this entire CRM, we're gonna need to build a front end and that front end will need to be a desktop version of website version and an app in the App Store for members to download and use just like they use the cross the hotel app for push press the crossroad Roel App has a main dashboard where it shows their profile picture. It shows the Jim Crossfit hotel. It has a gear icon for settings, where they can change your appearance change card bank information any notifications emergency contacts phone number email. It also shows check-in numbers and Lil KP KPI tile saying all time check-in committed club streak, and we'll talk more about all that later and then it has a schedule where they can reserve for classes check in for classes and then look at events and appointments, check-in reserve for appointments and events and then it also has a place for them to buy appointments such as an embody body composition, scan, and schedule that appointment so we're gonna need all of those features on the app and just all the backing features of that on the CRM and then it has a workout section on the on the tabs at the bottom and that workout section is where people can log their workouts and see the workouts for the day once again you can scrape the app later and you can look at it more detail so we'll need a place for all members to log their workouts and then to be able to see their everybody else's results to log a workout and then all the divisions like male RX female, RX, etc. and then for other people's workouts, they need other people need to be able to react to those people's workouts. They were going to need to be able to comment on those people's workouts leave comment GIFs so we'll need to have a Jim integration and a bunch of other things and then basically a member feature for all tax fields within there so they can like add a member in the comment section so that that member gets notified if they're not the one being left a comment on and then also on that home screen of the app that's where people can go in their app and look at all their past benchmarks their past workouts all of that other information and then the final page on the app is called the social page that's where anyone included. Anyone can leave a post and that post just goes to members like that we all have active logins into the app and that's where we make some gym kind of notifications. Sometimes members will leave a like a post in there. I want to build that whole feature and then on top of that what's not in the hotel App is I want to build an embedded Instagram feed feature just for our CrossFit OTL Instagram so that the people who do not have Instagram can still see all of our Instagram stories and our Instagram post directly in the OTL app that way they can always stand informed with our social media without having to have social media and then for the backend for the CRM backend I'll have all of our members in there you're gonna need to go scrape all of our push press logins, and we're gonna need to download and make sure that when we move members over that they stay properly tagged and on the correct part and on the correct campaign for emails and text etc. so that when we move them over they don't just get hammered with a bunch of new emails or anything like that so we need to make sure that that's a smooth process

Message 4 — 2026-09-19

Thecrossfit Dash otl.com domain is hosted on Go Daddy. My Mac mini already has access to that information right now you're kind of just notetaking doing research and then on our Mac mini will start developing and creating all of this and we create a new get help project a new Superbase project will create connect to the current cloud flare that host, my fleet, West and Classic Pro and Longhorn Longhorn Forge, and then we can connect it to the Go Daddy domain and all of the above, which will give us the ability to connect emails that way you know the app can auto send email to CRM. Can Auto send emails from that domain and then if needed we can also create a resend account that I have for MyFleetOS that way the CRM can send emails basically on our behalf that are like do not reply emails. They're just sending emails and and stuff like that the other thing that's kind of big that we need to add as well is there something called a StaffApp and so we have the CrossFit OTL app we have to push press software, but then we also have a push press app that's in the App Store that we sign into with our staff accounts, which are the same accounts as OTL member facing App and that Staff account allows us to check in members to appointments, events and classes. it also allows us to view all the information about that specific member and the bigger thing here is it's a point of sale system, which where we put all of our products and services such as element drinks, element, electrolyte packets, and body Scans. Anything you could possibly think of all that's all that's in our push press system right now online on the website will need all of those products transferred over as well and all of the statistics and past sales transfer over so that we have accurate data but in that point of sales system in the StaffApp, we are gonna need to be able to have a legit POS system right? I can charge somebody their card in their information from their member account is transferred over to the StaffApp so I can charge their bank account or card on file. I can search about people's names and I want to be able to make refunds from there, but those refunds need to be guided to an admin, and then you know whoever is listed as in legit Admin will be picked within the CRM by the Admin that way certain staff aren't eligible to make refunds, but certain Admins, who are listed as Admin for the POS can make refunds if needed.

Train.pushpress.com is the push press software for hosting and creating workouts that get pushed to our staff and member App. You'll need to create a log scrape it so I'll sign into train.push press that way you can see how that set up and we're gonna need to build that into our CRM and then with that we're gonna need our CRM to be able to create custom workouts all of the above and one thing that's a kind of a thing that's really hard to do on trying to push press system compared to what I want to build is really hard to change or edit how workouts are logged or what's listed on a workout without having to go all the way to your computer and go to trainer train the train app I wanna be able to just go into a Staff account or the StaffApp or whatever whatever that may be and maybe instead of having a crossword OTL app and then having a StaffApp like the current set up for push press, we have just a push press or we just have a CrossFit OTL app and if you are a staff or an admin, whatever that may be if you are a staff, there's a section that opens up on your crossword app and inside that section is where you have all the StaffApp things that way you're not having two abs it's not separate a lot simpler everything is just hosted on one app and it's just gate kept by if you are a staff or if you are not a staff, I think that's gonna be a lot better of a system than having two separate apps but what I'm saying with this is with the train app it is hard for somebody. It's hard for somebody. Who's in charge of programming a workout to make an edit to the workout if they do not have their computer open with they logged in train so if they are a staff and if they have access to the CrossFit app right if they are an admin allowed to do programming or their tagged as a programmer and by their member account or their Staff account whatever that may be, they can go into the CrossFit OTL app or obviously go into the CRM online that we're gonna be creating and they can just go and change the workout from there. Obviously, you can go through procedures like hey you know you have to enter a password maybe before you do it or before it officially changes or then if the people have already locked the out you're changing it it will give like a little bit of like a people already locked outs if you change this, they will get a push notification that you changed how the workout is formatted to go re-lock the workout and things like that but that way we can just edit workouts in App as the staff, of course that instead of going all over the place to try and do that and the same thing for the point of sales system the point of sales systems in that StaffApp why why don't we take the entire staff out and we move it all the way over to the CrossFit OTL app that we are gonna have to build and it's just gate kept by whether or not their staff and then certain permissions or be kept whether or not they're an admin. They're a programmer all of that good stuff.

So how would I get cap programming sent over to our CRM that we are going to build that's 100% separate than push press because we are ditching everything push press in building our own

Message 5 — 2026-09-19

Save EVERYTHING as long detailed pdf that includes un edited texts of what I have sent in chat and everything you have done and researched and found

And I am going to give you more links and such for you to save or file etc so that Mac mini has easy access to everything we have talked about so far

Can you read and write into my Nicks Vault in obsidian? If so you can just save it there too

Part II — What was researched and found

Method note

Where marketing pages and docs sites disagreed with implementation, implementation won. The endpoint inventory below was read from the **official SDK source code**, not from documentation. DNS findings come from live public DNS queries. Apple's guidelines are quoted verbatim from `developer.apple.com`.

1. PushPress Platform API — what exists and what doesn't

****Verified against @pushpress/pushpress v1.15.0 (npm, published 2026-04-15)**** by downloading the package and reading `src/funcs/`, `src/lib/config.ts`, `src/lib/security.ts` and `src/models/`.

Connection

Item	Value
Production base URL	<code>https://api.pushpress.com/v3</code>
Staging	<code>https://api.pushpressstage.com/v3</code>
Development	<code>https://api.pushpressdev.com/v3</code>
Auth header	API-KEY: <key> — **custom header, not Authorization: Bearer**
Env var SDKs read	<code>PUSHPRESS_API_KEY</code>
Scoping	Most operations take <code>companyId</code>
Pagination	1-based page + limit (SDK default 10)
Date filters	Check-in endpoints accept <code>after</code> / <code>before</code>
SDKs	@pushpress/pushpress (TS, 47 versions) · pushpress/php-sdk

Where the key comes from

Core -> Settings -> Security & Access -> Add API Key (admin required). Self-serve. There is nothing to request from PushPress and no pre-existing key hidden in the account.

Three distinct keys exist and are **not interchangeable**:

- **Core Platform API key** — the one that matters. Settings -> Security & Access.
- **Grow API key** — Grow CRM only (marketing side). Separate product, separate key.
- **Developer portal key** — `developer.pushpress.com`, login-walled; same API, also mintable via `/keys`.

Key management endpoints

`POST /keys` (accepts name, description, expiresAt) · `GET /keys` · `GET /keys/{id}` · `POST /keys/{id}/revoke` · `DELETE /keys/{id}`

Complete endpoint inventory — 40 paths

Customers: `/customers` (GET, POST) · `/customers/{uuid}` · `/attributions/attributions` · `/attributions/attributions/{uuid}`

Check-ins: /checkins/class · /checkins/class/{uuid} · /checkins/appointment · /checkins/appointment/{uuid} · /checkins/event · /checkins/event/{uuid} · /checkins/open · /checkins/open/{uuid} · /checkins/count

Scheduling: /classes · /classes/{id} · /classes/types · /classes/types/{id} · /events · /events/{id} · /appts/{id} · /reservations · /reservations/{id}

Membership & billing: /enrollments · /enrollments/{uuid} · /plans/{id} · /company · /invitations · /invitations/{id}

Messaging: /messages/email/send · /messages/sms/send · /messages/push/send · /messages/notification/send

Keys & webhooks: /keys · /keys/{id} · /keys/{id}/revoke · /webhooks · /webhooks/{uuid} · /webhooks/{uuid}/activate · /webhooks/{uuid}/deactivate · /webhooks/{uuid}/rotate-signing-secret

Webhook events — all 23

app.installed · app.uninstalled · appointment.canceled · appointment.noshowed · appointment.rescheduled · appointment.scheduled · checkin.created · checkin.deleted · checkin.failed · checkin.updated · class.canceled · customer.created · customer.deleted · customer.details.changed · customer.status.changed · enrollment.created · enrollment.deleted · enrollment.status.changed · memberapp.updated · reservation.canceled · reservation.created · reservation.noshowed · reservation.waitlisted

The finding that shapes everything

Zero endpoints, zero data models and zero webhook events reference workout, result, score, benchmark, rep max or PR. A case-insensitive grep across the entire SDK source returns nothing. Also absent: products, orders, invoices, transactions — so POS data has no API either.

The API covers the **business**. It does not cover **performance**.

2. The CAP -> Train credential — whose it is

The initial assessment that the sync uses "no API key" was wrong, and was corrected. There *is* a credential. It belongs to **PushPress the company**, held server-side under a business arrangement with CrossFit. It is never issued to an affiliate, appears nowhere in Core or Train settings, and has no gym-level equivalent to generate.

Evidence it is platform-level, not per-gym: CrossFit's own CAP page states programming is delivered to **Google Docs, SugarWOD, Wodify, PushPress, STREAMFIT, BTWB, Strivee, WODBoard and Chalk It Pro** — nine separate platform integrations, i.e. nine commercial arrangements. CrossFit publishes **no API, no feed specification and no developer access** for CAP anywhere.

So "create our own key" would mean becoming a CAP distribution partner: a business conversation with CrossFit, not a credential to mint.

What CAP actually includes

- **Included with affiliation** — "No extra subscription. No extra cost. Just access."
- Requires a CrossFit affiliate in good standing; the Train-delivered version also needs a Train subscription
- Delivered weekly, next week's released **by Friday at the latest**
- Daily workouts, scaling options, full class plans with coaching notes, warm-ups, timelines
- Tracks: standard CAP plus an optional **Compete Track**; SugarWOD's materials also reference

Affiliate, At Home, Compete and Lifting tracks

- Core <-> Train **member sync is a separate opt-in toggle** — without it, results don't line up against member records

3. SugarWOD — the control case

Checked because if another CAP partner exposed results, switching platforms would solve the gap. **It doesn't.**

Item	Value
Base URL	https://api.sugarwod.com/v2
Auth	Authorization: <KEY> — bare key, **no Bearer prefix**
Alternative	?apiKey=<key> query param
Key creation	Self-serve, SugarWOD portal /gyms/settings/developer-keys

Exposes: /box, /box/account-owner · /athletes (+ /{id}, /find?email=, POST, PATCH, /{id}/remove) · /athletes/{id}/summary/performance, /summary/participation · /workouts?dates=&track_id=, /workouts/{id}, /workouts/{id}/athletes · **Workouts HQ** (CrossFit mainsite programming) · /benchmarks/{id}, /benchmarks/category/{category} · /barbelllifts/{id} · /movements, /movements/{id} · /tracks (GET, POST) · webhook event.affiliate.ATHLETE_JOINED

Does not expose: individual athlete scores. /workouts/{id}/athletes returns *who logged*, not what they scored. Both summary endpoints are published but marked "**not yet available.**" No CAP/mainsite athlete results.

Conclusion: the results gap is an industry norm, not a PushPress quirk. Switching platforms does not fix it.

Worth noting SugarWOD does expose things PushPress v3 doesn't: workout programming, benchmarks, barbell lifts and movements as first-class objects. Useful as a reference vocabulary when modelling our own schema.

4. Stripe — the best news in the project

Account ownership

PushPress's own help documentation: **the Stripe account is "typically under the gym owner's email from when PushPress was set up."** Setup path is Core -> Settings -> Billing & Payment -> "Connect with Stripe" -> Stripe onboarding, approval typically instant. Gym owners get **direct Stripe Dashboard access**; the docs instruct users to "Log in to dashboard.stripe.com," stating "Your payout bank account is managed in Stripe directly."

Implication: if this holds for OTL, there is no card or bank migration at all. You keep the account. The new CRM authenticates to the same account with your own keys. Every cus_... and pm_... stays put. Nobody re-enters anything.

The real risk is schedules, not cards

Open the Subscriptions tab in the Stripe Dashboard:

- ****Live sub_ objects exist**** -> billing schedules live in Stripe and largely survive. Rebuild the UI, not the billing.
- **Only one-off PaymentIntents** -> PushPress runs its own recurring biller. More likely, given their

API models /enrollments and /plans as first-class objects. **When you leave, recurring billing simply stops**, and the new CRM must recreate every schedule exactly: amount, interval, next charge date, proration, discounts, comps, paused states, family plans, annual vs monthly.

This is the single highest-risk item in the project. Wrong one way, revenue stops. Wrong the other, members get double-charged.

Fallback if the account turns out platform-owned

Stripe's **self-serve PAN copy**, available in the Dashboard:

- Copies **Customers, Cards, Sources, Payment Methods and Bank Accounts**
- **Preserves original Customer IDs** — foreign keys survive
- **Subscriptions are NOT copied** — recreate via Dashboard or API
- Sender and recipient exchange acct_ IDs from User settings
- Accounts are rate-limited per period; Stripe's Data Migration team handles larger jobs
- Requires the sending account's cooperation
- Three variants: full copy, partial by customer selection, partial by CSV upload

Stripe also documents PAN **import** from another processor and PAN **export** to one. For any migration they require the request to include **both customer records and the associated payment data**.

Hard rules

- **Never let a PAN or bank number touch our servers** — not in a database, log, scraped blob, or

"temporarily." Storing one puts us in **PCI-DSS scope**, an audited compliance programme. New cards come in via Stripe Elements (web) / PaymentSheet (mobile); the backend only ever sees pm_ and cus_.

- **ACH mandates:** bank debits carry a recorded authorisation tied to a merchant and statement

descriptor. Confirm with Stripe that existing mandates survive if anything changes. A changed descriptor produces disputes from members who don't recognise the charge.

- **Do not consolidate OTL into another entity's Stripe account.** Different legal entity = different merchant of record. The authorisations members gave were given to CrossFit OTL.

5. Apple App Store — quoted verbatim

Retrieved from developer.apple.com/app-store/review/guidelines/.

> **3.1.3(e) Goods and Services Outside of the App:** "If your app enables people to purchase physical > goods or services that will be consumed outside of the app, you must use purchase methods other than > in-app purchase to collect those payments, such as Apple Pay or traditional credit card entry."

> **3.1.3(d) Person-to-Person Services:** "If your app enables the purchase of real-time > person-to-person services between two individuals (for example tutoring students, medical > consultations, real estate tours, or fitness training), you may use purchase methods other than > in-app purchase to collect those payments. One-to-few and one-to-many real-time services must use > in-app purchase."

> **3.1.1 In-App Purchase:** "If you want to unlock features or functionality within your app, (by way > of example: subscriptions, in-game currencies, game levels, access to premium content, or unlocking > a full version), you must use in-app purchase. Apps may not use their own mechanisms to unlock > content or functionality..."

Reading for OTL: memberships, in-person classes and InBody scans are services consumed in the real world at the gym -> **3.1.3(e) prohibits IAP.** Stripe is not merely permitted, it is required. **No 30% cut.**

Where 30% would apply: anything digital — an on-demand video library, remote programming, a premium app tier -> 3.1.1, IAP required. Also note one-to-many *real-time virtual* sessions fall under 3.1.3(d) and require IAP; in-person classes do not.

6. Instagram — half the feature is achievable

- **Instagram Basic Display API** was permanently shut down on 4 December 2024. All requests to its endpoints now error. Anything built against it is dead.

- Replacement is the **Instagram Graph API**, fully supported.
- **Only Professional accounts** — Business or Creator — are supported. Personal accounts are not.

Confirm OTL's account type before promising this feature.

- **Own posts and reels** are readable. This part is fine.
- **Stories** require elevated permissions and Meta **app review**, and are only available for 24 hours. Meta's Platform Terms also restrict caching and storing content.

Plan: ship the posts feed; treat stories as contingent on app review, and don't announce them to members until Meta approves.

7. crossfit-otl.com — live DNS audit, 2026-09-19

Queried over DNS-over-HTTPS against Cloudflare's resolver.

Record	Value	Meaning
MX	0 crossfitotl-com02b.mail.protection.outlook.com	Mail on Microsoft 365
NS	ns21.domaincontrol.com, ns22.domaincontrol.com	DNS still at GoDaddy , not Cloudflare
TXT (root)	google-site-verification=Ff0s2dAuKx0auNPNI3J-ktZI8ZbcGMH46iA-2u6WrU only	No SPF
TXT _dmarc	<i>absent</i>	No DMARC
TXT selector1._domainkey	<i>absent</i>	No DKIM (M365 default selector)
CNAME www	e106509a8f5ab8ba.vercel-dns-017.com	Website already on Vercel

Why this matters immediately

- Staff email is **already** sending unauthenticated — weaker deliverability, and ****anyone can spoof @crossfit-otl.com today.**** This is a live exposure, independent of the CRM project.
- If the CRM starts sending from this domain as-is, **it lands in spam**. Gmail and Yahoo have required authentication from bulk senders since early 2024.

Two things already align with the plan

The website is on Vercel and the domain is in hand at GoDaddy. Nothing to migrate there.

8. PushPress export & reporting routes — better than expected

The v3 API has no products or sales endpoints, **but Core's reporting does**, as CSV and Excel:

Report	Path	Contents
Retail Sales — by Category	Reports -> Financial -> Retail Sales	Product sales by category (summary)
Retail Sales — Detail	same	Every product purchase and miscellaneous charge
Financial Details	Reports -> Financial	Every transaction: payments, refunds , discounts, fees. Best for audit and bookkeeping
Daily Deposit	Reports -> Financial	Deposit reconciliation
Member list	Members -> Download Members	First/last name, email, address, city, state, postal, phone, plan(s), per-plan status (active, completed, canceled, paused, alert)

Download via the ... menu -> **Download Data**. **Critical:** set **Advanced Data Options -> All Results** for large date ranges, or the export is silently truncated to a subset.

Also noted: PushPress only accepts CSV for *imports*, and documents importing Train workout history from SugarWOD and Wodify — evidence that results import is possible in their system even though results export is not exposed.

9. Stripe Terminal / Tap to Pay on iPhone — POS requirements

For the staff POS, verified from Stripe's Terminal documentation:

- Official **React Native SDK**: stripe/stripe-terminal-react-native
- **Apple entitlement required** — request the **development** entitlement, then a ****separate**

distribution **entitlement after internal testing. Neither is instant.** Start this early.**

- Device floor: **iPhone XS or later**, iOS **15.1+** for the React Native SDK
- Apple **mandates** a "How to Tap" instructional overlay via the ProximityReaderDiscovery API,

integrated **before** submitting for review. ProximityReader is a built-in iOS framework already linked by the Stripe SDK — no extra dependency.

- **Location services must be available** or Stripe disables payments outright, for fraud and dispute

reasons

- Accepts Visa, Mastercard, Amex contactless plus Apple Pay, Google Pay, Samsung Pay

PushPress already offers Stripe Tap to Pay and Stripe readers through its staff app, on the same Stripe account — so this is a like-for-like replacement, not a new capability to sell internally.

Part III — The plan

A. Migration

Data inventory and route for every dataset

Data	Route	Notes
Member profiles, contact info	API GET /customers	CSV "Download Members" as cross-check
Emergency contacts	[?] Verify — may be CSV or app-only	Likely a scrape target
Profile pictures	[?] Check if API returns URLs	Else pull from app JSON
Tags / segments	Grow API or CSV	Critical — drives campaign safety
Leads / prospects	Grow CRM (separate key)	Future revenue; don't forget
Enrollments & plans	API /enrollments, /plans/{id}	The billing schedule source
Check-in history	API /checkins/* (4 types)	after/before filters
Classes, events, appointments	API /classes, /events, /appts/{id}	
Reservations & no-shows	API /reservations	
Committed Club, streaks, all-time counts	Derive from check-ins	Don't copy the number — recompute
Cards & bank accounts	Stripe, never PushPress	See §4
Payment / invoice history	Stripe API	Stripe is source of truth
Products & services	[OK] Core -> Retail Sales export	Supported CSV/Excel
Sales & POS history	[OK] Core -> Financial Details export	Payments, refunds, discounts, fees
Sales tax config	Core settings — carry deliberately	Don't re-derive
Programming / WODs	CAP via Google Docs , our credentials	See §D
Results, benchmarks, PRs	No supported export	Leaderboard export + app scrape
Comments, reactions, posts	No export	Decide: migrate, archive, or start fresh

Do this before giving PushPress any notice

Access dies with the account, and results have no supported path out. Order of attack:

- **Ask PushPress in writing** what export departing customers get. They run a migration team for inbound data — ask what's available outbound. Get it documented.
- **Leaderboard export** from Core. Check whether rows carry member, workout, date, score and input type. If structured, it may suffice alone.
- **Capture the app's own API responses** — last resort, best form of it.

On scraping — do it well or not at all

It's our gym's data and extracting it is legitimate. Two practical cautions:

- **Check PushPress's terms on automated access** before running at scale. A supported export waited a week for beats a scraper maintained forever.
- **Capture JSON, not HTML.** The OTL iOS app talks to a backend in JSON. A proxy (mitmproxy, Charles)

against the app on the mirrored iPhone records typed, structured data — dates as dates, scores with units and scaling intact. DOM scraping yields formatted strings to re-parse, and breaks on every deploy.

Setup at the Mac mini: Playwright or Chrome DevTools Protocol driving the already-logged-in Chrome profile for Core; mitmproxy against the mirrored iPhone for Train and member-app surfaces.

Don't paste passwords into chat

Chat history persists. Point automation at an already-authenticated browser profile and pass API keys as environment variables. Driving a logged-in session never requires seeing a password.

Reconciliation — definition of done

Not "the script ran."

- Member count in extract == member count in Core UI
- Active enrollments == active members in Core billing
- Per-member check-in totals == the all-time number the app shows that member
- Ten members spot-checked by hand across every field
- Every extract archived off the working machine, encrypted

Comms safety — migrating without spamming everyone

Rule: import suppressed, verify, then enable.

- **A global send kill switch**, not per-contact flags. One system-level flag blocking all email, SMS and push. Off by default. Per-contact flags get missed on one code path and that's enough.
- **Preserve state, not just membership**: tag membership, campaign enrolment, ****current step position in each sequence****, and consent/opt-out status.
- **Opt-outs are legally load-bearing**. Re-subscribing someone who unsubscribed is a CAN-SPAM problem for email and a **TCPA** problem for SMS — the latter carries statutory damages per message. Opt-out status migrates first and is verified by hand.
- **Never let "contact created" fire on import**. Every welcome sequence triggers on creation. Disable those automations during import, or gate as created `AND source != 'migration'`.
- **Stamp every record** with `migrated_at` and `migration_source`, then guard every automation: don't fire for events predating `migrated_at`. This single rule catches most of what the others miss — it stops re-celebrating three years of old PRs and re-sending two years of birthday emails.
- **Ramp the test**: 5 real staff accounts -> flip sends on -> watch. Then 50. Then everyone.
- **Quiet period**: sends off 24–48h after go-live while reconciling.
- **One deliberate announcement**, from you, before cutover. What's changing, what they need to do (ideally nothing), what their statement will say, and that their PR history came with them.

Phased cutover

Phase	What moves	Risk	Reversible
0. Extract	Nothing. Pull and verify.	None	n/a
1. Mirror	CRM ingests PushPress read-only, on a schedule	None	Yes

Phase	What moves	Risk	Reversible
2. App v1	Members log workouts, view schedule	Low	Yes
3. Scheduling	Reservations and check-in	Medium	Yes, painfully
4. Billing	Recurring charges. Alone.	High	Barely
5. Extras	Social, Instagram, appointments, marketing	Low	Yes

Never cut over billing and the app on the same day. Every other mistake is a bad week; billing failure is lost revenue and cancellations.

Cutover day (Phase 4): PushPress biller **confirmed stopped** (verified, not assumed) · new biller's first run executed in Stripe **test mode** against real schedules and reconciled line by line · every active enrollment matched on amount, interval, next date, discounts, paused/comped · statement descriptor checked · sends still suppressed · rollback defined in advance with a named decision-maker · first live cycle reconciled **by hand**, member by member, a full day budgeted.

Timing: cut over **immediately after** a billing cycle completes, not before one — that buys a full cycle of runway. Avoid the week of the Open and holidays.

B. Architecture

Stack

Layer	Choice	Why
Database	Supabase (Postgres)	Relational fits; adds auth, RLS, storage, realtime
Auth	Supabase Auth	Email, Apple, Google. Apple sign-in effectively required for iOS social login
File storage	Supabase Storage	Profile pictures, post images
Realtime	Supabase Realtime	Comments, reactions, social feed update live
Web + admin	Next.js on Vercel	Member web app and staff/admin CRM. www already on Vercel
Mobile	React Native + Expo	One codebase iOS+Android, shares types with web, OTA updates
Payments	Stripe — Elements / PaymentSheet / Terminal	Existing OTL account
Push	Expo Push -> APNs/FCM	
Human email	Microsoft 365	Existing, root domain
Machine email	Resend	On a <code>notify.</code> subdomain
GIFs	Giphy or Tenor API	Workout comments

Domain model

```

Member --< Enrollment >-- Plan
|
+---< CheckIn >-- ClassInstance >-- ClassTemplate
+---< Reservation >-- ClassInstance
+---< Result >-- Workout >-- WorkoutTrack (CAP, Compete, ...)
+---< PR >-- Movement
+---< Appointment >-- AppointmentType (InBody, PT, ...)
+---< Post / Comment / Reaction >
+---< MemberTag >-- Tag
+---< CampaignEnrollment >-- Campaign
\---< Sale >-- SaleLineItem -- Product

```

Decisions worth making now, because they're expensive later:

- ****Result needs ScoringType, Division and a scaling flag**** — not a bare number. A result is a value *plus* how it was scored, in which division, scaled or not, sometimes with a tiebreak.
- ****Movement is first-class****, so PRs and benchmarks resolve against a controlled vocabulary rather than free-text workout names.
- **Derived data stays derived.** Streaks, Committed Club and all-time counts are a materialized view over CheckIn, refreshed nightly. Stored numbers drift, and a drifted streak is a support ticket.
- **Keep external IDs on every record** — pushpress_id, stripe_customer_id. Months of reconciliation ahead, and re-running an import without them creates duplicates.
- ****migrated_at / migration_source on every imported row.****

Billing engine

Prefer Stripe Subscriptions over a home-grown scheduler. Stripe handles retries, dunning, proration, SCA and failed-payment recovery — all of which get rebuilt badly otherwise. The CRM stores membership *policy*; Stripe executes *billing*. Custom concepts (holds, comps, family plans, class packs) map **onto** a Stripe subscription rather than replacing its scheduler.

Handle webhooks idempotently: invoice.payment_failed, invoice.paid, customer.subscription.updated, payment_method.attached. Idempotency keys on everything outbound.

Security baseline

Non-negotiable for a system holding member PII, possibly minors' data, emergency contacts and payment tokens: no PANs ever · secrets in the platform secret store, separate keys per environment · audit log on every staff action touching member data or billing · encrypted, **restore-tested** backups · MFA on every staff account · export and deletion paths built early for privacy requests.

One caution on row-level security

RLS is excellent and easy to get subtly wrong. A member must never read another member's contact details, payment info or emergency contacts — but *must* read their workout results and comments. Write policies deliberately and test with a real non-admin session, never from the service-role key.

C. One app, role-gated — staff mode and POS

PushPress splits member and staff into two apps. **Collapsing both into one CrossFit OTL app with staff features gated by role is the right call:** one codebase, one auth system, one release cycle, one App Store listing, and staff stop carrying two apps. Accounts are already shared, so nothing is lost.

Roles — per-capability, not a boolean

Role	Can do
member	Own profile, schedule, workouts, social
staff	Check members into classes/events/appointments; view member details; take POS sales
programmer	Create and edit workouts, publish CAP parses
admin	Member management, plans, enrollments, settings
pos_admin	Issue refunds. Explicitly assigned by an admin in the CRM

Additive, assigned in the CRM web admin, never self-served in the app. Every role-gated action writes an audit entry: who, what, when, which member, how much.

Apple can't see gated features — supply a demo account

A reviewer logging in as a plain member sees none of the staff surface, and "unable to locate the described functionality" is a routine rejection. **Include a staff demo account in App Review notes on every submission.** Consider a demo-mode flag exposing staff UI against seeded data.

POS — three payment paths, all on the existing OTL Stripe account

- **Card present — Tap to Pay on iPhone.** Stripe Terminal React Native SDK. Requirements and Apple entitlement lead time in §9 above. Replaces the reader/Tap to Pay already used via PushPress.
- **Card on file** — the most-used path in a gym. Search a member by name, charge the saved card or bank account. Mechanically an **off-session PaymentIntent** against the saved pm_. Because the member isn't present to authenticate this is a **merchant-initiated transaction** and must be set up correctly *at save time* or these charges fail authentication. Get this right early.
- **Manual entry** — fallback for a member with no saved method. Elements / PaymentSheet. Never a raw PAN in our UI.

Refund approval workflow

```
staff initiates -> refund REQUEST (pending) -> pos_admin reviews -> Stripe refund -> audit log  
  \ declined, with reason
```

- Staff **cannot** refund. They raise a request with amount and reason.
- Only pos_admin approves; that role is assigned by an admin in the CRM.
- Approver gets a push notification; requester gets the outcome.
- Every request, approval and decline logged immutably.
- Partial refunds supported; refund can never exceed the original charge — enforced server-side.
- **Idempotency keys** on every Stripe refund call so a double-tap can't double-refund.

Make the request the only path, even for admins

An admin refunding directly, bypassing the request record, is how audit trails develop holes. Let admins approve their own requests instantly if you like — the record still gets written.

Products and inventory

Model: Product (name, category, price, taxable, active) · InventoryItem (stock, reorder point) · Sale -> SaleLineItem · Payment -> Stripe charge · Refund -> Stripe refund + approval record. Everything currently in PushPress: Element drinks and electrolyte packets, body scans, drop-ins, merch, services. Extraction route in §8. Cross-check totals against Stripe.

Editing workouts from the app

The pain point named: changing a workout today needs a computer and a Train login. In the new system a programmer edits from the app or the CRM.

Edits after results exist must version, not overwrite

A destructive edit silently invalidates logged results and corrupts benchmark history.

- Workout carries a version; an edit after the first logged result creates a new version
- Existing results stay bound to the version they were logged against
- Everyone who already logged gets a **push notification** naming what changed, with a prompt to re-log
- Require re-authentication or a confirmation step for edits to a workout with results
- Show the programmer the logged-result count **before** they confirm: "14 members have logged this."

D. CAP programming into a fully independent CRM

The route is **Google Docs**, and it's the one already under our control. Of the nine CAP delivery targets, eight are platforms being replaced. Google Docs is the ninth — affiliate-facing, no platform in between. And CAP belongs to the **affiliation**, not to PushPress, so leaving costs nothing.

```
CrossFit -> our Google Doc -> Drive/Docs API -> parser -> staff review -> published Workout
          (CAP delivery)   (our OAuth)   (LLM)   (programmer)   (CRM + app)
```

- **Detect** — poll the Drive folder / doc for the weekly drop (released by Friday at the latest)
- **Fetch** — Google Docs API, our own OAuth credentials
- **Parse** — doc text -> typed records
- **Review** — a programmer confirms before it goes live
- **Publish** — a workout on the CAP workoutTrack, visible in app

Parsing is the actual work. CAP arrives as prose written for coaches — workout, scaling, coaching notes, warm-up, timeline — not structured data. LLM structured extraction is a genuinely good fit: document text in, typed JSON out against our schema, far more robust than regex against prose whose format shifts week to week.

Never auto-publish an unreviewed parse

A misparsed rep scheme or time cap goes straight onto the whiteboard and into members' logged results. Human review is required, not a nicety. Show the parse beside the original text so the programmer can eyeball it in seconds. **Keep the raw source text on every workout record** so a bad parse can be re-run without re-fetching a doc that may have moved on.

Routes that don't work: using PushPress's CAP credential (theirs, server-side, never issued to affiliates) · becoming a CAP distribution partner (a business deal with CrossFit; unrealistic for one gym and unnecessary) · scraping Train permanently (works, but keeps us dependent on what we're leaving).

Interim bridge: during Phases 1–3, scraping Train for programming is legitimate — it gets the app working before the Google Docs pipeline exists. Just don't let it become the permanent answer.

Licensing: CAP is CrossFit's IP, licensed to affiliates in good standing. Ingesting it to show *our own members* is the intended use. Don't republish publicly or expose it to non-members.

E. Infrastructure — domain, DNS, email

Email architecture

Send CRM mail from a subdomain, never the root

notify.crossfit-otl.com via Resend, with its own DKIM. This isolates reputation: any system mailing hundreds of members eventually generates complaints, and those must damage the **subdomain's** reputation, not the root domain staff email depends on.

Root crossfit-otl.com -> Microsoft 365, human staff mail. Subdomain notify. -> Resend, machine mail, do-not-reply.

Setup order: SPF on root for M365 -> DKIM on root for M365 (both selectors) -> ****DMARC at p=none**** with an aggregate report address -> subdomain delegated to Resend with its own SPF/DKIM/DMARC -> monitor reports for a few weeks -> root policy to quarantine, then reject.

Don't jump to p=reject

You will discover forgotten services legitimately sending as the domain. p=none with reporting identifies them before their mail starts bouncing.

Moving DNS to Cloudflare

Carry MX, TXT and the Vercel CNAME across explicitly

Cloudflare's import scan usually catches existing records, but **verify MX by hand before flipping nameservers**. A missed MX means email silently stops and it may be hours before anyone notices.

Pre-switch checklist: MX -> Outlook · the google-site-verification TXT · www -> Vercel · any Microsoft autodiscover/verification records · the new SPF/DKIM/DMARC. Mail records must be **DNS-only (grey cloud)**, never proxied.

F. Scope — the honest number

12–24 months for a small experienced team

Full gym CRM, billing engine, scheduling, workout tracking with leaderboards and PRs, social feed with reactions/comments/mentions, POS, marketing automation, a web app, a native iOS app, an admin console, **and** a zero-downtime migration of live members and live billing. Not a Mac mini weekend.

That isn't a reason not to do it — it's a reason to sequence it so every phase is useful alone and nothing irreversible happens early.

PushPress is four products in a trench coat: Core (CRM, billing, scheduling), Train (programming and results), Grow (marketing automation), and a member app — plus a staff app and POS. Each is a company. Wodify, SugarWOD and PushPress each took years and funded teams.

The leverage: it only has to serve one gym

No multi-tenancy, no per-gym configurability, no migration tooling for other people's data, no billing flexibility beyond what OTL actually sells, no enterprise support. That strips out most of what makes gym software expensive — and it's the reason this is achievable at all. Resist every urge to build it "properly" for gyms in general.

Phase	Deliverable	Rough effort	Value if you stop here
0	Extract + verify all data	Days	Insurance. The data is yours regardless.
1	Read-only mirror in our schema	2–4 weeks	Proves the data model against live data. Zero risk.
2	Member app v1 — schedule + workout logging	2–4 months	Members using our app daily. The real adoption test.
3	Scheduling + check-in cutover	1–2 months	PushPress becomes billing-only
4	Billing cutover	1–2 months + reconciliation	PushPress cancelled
5	Social, Instagram, POS extras, marketing	3–6 months	Parity and beyond

Decisions that matter most: don't cancel PushPress until Phase 4 is reconciled — a few hundred a month of parallel running is the cheapest insurance on this project · **Phase 2 tests members, not code** — if they won't adopt the app while PushPress still works, they certainly won't after billing moves · **billing is the only phase that can damage the business** · **resist rebuilding marketing automation** — Grow's replacement is the least differentiated, most fiddly piece; consider syncing members to an off-the-shelf tool instead of building sequences, templates and deliverability.

Part IV — Action checklist

This week, independent of everything else

- ****Add SPF, DKIM and DMARC to crossfit-otl.com.**** The domain is spoofable today and blocks all CRM email until fixed. Cheapest, highest-value item on this list.
- **Verify Stripe account ownership** — log into `dashboard.stripe.com` with the gym owner email. Confirm OTL's live payments are visible. Note the `acct_` ID. 30 minutes; determines the entire payment plan.
- **Check the Stripe Subscriptions tab.** Live `sub_` objects, or only one-off `PaymentIntents`? This is the single most consequential unknown in the project.
- **Ask PushPress in writing** what data export departing customers receive. Get it documented.
- **Request the Apple Tap to Pay entitlement** — development first, then distribution. Real lead time; nothing else unblocks it.
- **Do not cancel anything** until extracts are verified complete.

At the Mac mini

- Mint a Core API key: **Settings -> Security & Access -> Add API Key**. Store in a secret manager.
- Smoke-test: `GET /customers` and `GET /checkins/class` with `API-KEY` header and `companyId`
- Run `pushpress_extract.py`; reconcile counts against the Core UI
- Export from Core: **Members · Retail Sales (Detail) · Financial Details** — with

Advanced Data Options -> All Results

- Pull one **leaderboard export**; assess whether rows are structured enough to seed the results schema
- Sign into `train.pushpress.com` and walk the programming UI
- Mirror the iPhone; walk the OTL app screen by screen against the feature spec
- Point mitmproxy at the app and capture its JSON responses for results, benchmarks, profile photos
- Locate the **CAP Google Docs delivery** in the Affiliate Toolkit; determine the mechanism
- Email `affiliatesupport@crossfit.com` about structured CAP delivery for affiliates
- Confirm CAP is on in Train, and the **Core <-> Train member sync** toggle is enabled
- Confirm the Instagram account is **Business or Creator**, linked to a Facebook Page
- Register a `checkin.created` webhook; verify delivery and signature validation

Then, to build

- New GitHub repo · new Supabase project · Vercel project
 - Move DNS to Cloudflare — **verify MX by hand first**
 - Delegate `notify.crossfit-otl.com` to Resend
 - Phase 1: read-only mirror of PushPress data in our schema
-

Part V — Open questions blocking decisions

The first one changes almost every architectural decision. Answer it first.

- **Is this OTL's own tool, or a product to sell to other gyms?** Multi-tenancy, configurability and per-gym billing are most of what makes gym software expensive. Building for one gym strips them out.
 - **Does PushPress bill through Stripe Subscriptions or its own scheduler?** Determines whether billing cutover is a UI rebuild or a full scheduler rebuild.
 - **Is a member app on top of PushPress-as-system-of-record enough?** Phase 2 without 3–5 gets the Instagram feed, better social and better workout UX, and never touches billing. The blocker is the same results-API gap. Whether that's enough depends on what's actually driving the move.
 - **What is the actual driver — cost, missing features, control, or frustration?** If cost, compare honestly against 12–24 months of build time.
 - **Who maintains it at 6am when billing fails and you're coaching a class?**
 - **Exact CAP Google Docs delivery mechanism** — shared doc, Drive folder, or a new doc weekly?
 - **Rebuild marketing automation, or sync to an off-the-shelf tool?**
 - **Do comments, reactions and social posts migrate, get archived, or start fresh?**
-

Part VI — Sources

PushPress API & SDK

- <https://www.npmjs.com/package/@pushpress/pushpress> — official TS SDK (primary source for endpoints/auth)
- <https://github.com/PushPress/pushpress-ts>
- <https://packagist.org/packages/pushpress/php-sdk>
- <https://github.com/api-evangelist/pushpress> — third-party API surface profile
- <https://ppe.apidocumentation.com/> — API reference (root 404s to plain fetch)
- <https://developer.pushpress.com/> — developer portal, login-walled

PushPress help centre

- <https://help.pushpress.com/en/articles/12631598-how-to-use-pushpress-integrations-hub> — API key path
- <https://help.pushpress.com/en/articles/508482-core-connect-your-stripe-account> — **Stripe ownership**
- <https://help.pushpress.com/en/articles/508570-core-migration-exporting-member-data>
- <https://help.pushpress.com/en/articles/508575-core-retail-sales-report>
- <https://help.pushpress.com/en/articles/508582-how-to-use-the-financial-details-report-in-pushpress-core>
- <https://help.pushpress.com/en/collections/11657378-workout-results>
- <https://help.pushpress.com/en/collections/3275234-migration>

CrossFit / CAP

- <https://www.crossfit.com/affiliate-toolkit/crossfit-affiliate-programming> — included with affiliation
- <https://www.crossfit.com/affiliate-toolkit>
- <https://affiliate.crossfit.com/tools/programming/>
- <https://www.pushpress.com/partners/crossfit-affiliate-programming-x-train-by-pushpress>
- <https://www.sugarwod.com/2023/05/crossfit-affiliate-programming-sugarwod/>
- Contact: affiliatesupport@crossfit.com

SugarWOD

- <https://app.sugarwod.com/developers-api-docs>

Stripe

- <https://docs.stripe.com/get-started/data-migrations/overview>
- <https://docs.stripe.com/get-started/data-migrations/pan-copy-self-serve> — **self-serve PAN copy**
- <https://docs.stripe.com/get-started/data-migrations/pan-import.../pan-export>
- <https://support.stripe.com/questions/request-a-data-migration>
- <https://docs.stripe.com/terminal>
<https://docs.stripe.com/terminal/payments/setup-reader/tap-to-pay>
- <https://github.com/stripe/stripe-terminal-react-native>
- <https://stripe.com/customers/pushpress>

Apple

- <https://developer.apple.com/app-store/review/guidelines/> — 3.1.1, 3.1.3(d), 3.1.3(e)

Instagram / Meta

- Basic Display API shut down 2024-12-04; Instagram Graph API is the replacement
-

Appendix A — `pushpress_extract.py`

Read-only extractor for everything the documented v3 API exposes. Lives beside this document at `Migration/pushpress_extract.py`. Endpoints and auth verified against SDK source; pagination is 1-based page + limit; handles 429/5xx with backoff and Retry-After; safe to re-run (finished collections skipped unless `--force`); writes a manifest of counts for reconciliation.

```
export PUSHPRESS_API_KEY=...      # Core -> Settings -> Security & Access -> Add API Key
export PUSHPRESS_COMPANY_ID=...
python3 pushpress_extract.py --out ./extract
```

Collections pulled: `customers` · `enrollments` · `classes` · `class_types` · `events` · `reservations` · `invitations` · `checkins_class` · `checkins_appointment` · `checkins_event` · `checkins_open` · plus the company singleton.

Explicitly **not** covered, by design, because no endpoint exists: `workout results/benchmarks/PRs` · `cards and bank accounts (Stripe)` · `tags, campaigns and leads (Grow)` · `profile photos and social content`.

Appendix B — Vault file map

```
vault/6-Projects / OTL Platform/
+--- OTL Platform MOC.md          <- start here
+--- OTL Platform — Master Brief.md  <- this document
+--- OTL Platform — Master Brief.pdf
+--- Incumbent — PushPress/
|   +--- Incumbent — PushPress MOC.md
|   +--- PushPress API — Keys & Auth.md
|   +--- PushPress Platform API — Endpoint Inventory.md
|   +--- CrossFit Affiliate Programming -> Train Sync.md
|   +--- Workout & PR Data — Access Options.md
|   \--- SugarWOD API — Industry Comparison.md
+--- Migration/
|   +--- Migration MOC.md
|   +--- Data Inventory & Extraction Routes.md
|   +--- Stripe & Payment Continuity.md
|   +--- Cutover Plan & Comms Safety.md
|   \--- pushpress_extract.py
\--- Build/
    +--- Build MOC.md
    +--- Architecture — System Design.md
    +--- Member App — Feature Spec.md
    +--- Staff Mode & POS — Roles, Terminal, Refunds.md
    +--- CAP Programming — Independent Ingestion.md
    +--- Infrastructure — Domain, DNS & Email.md
    \--- Scope & Phasing — Honest Estimate.md
```

Related

- [OTL Platform MOC](#) — parent
- [Migration MOC](#) · [Build MOC](#) · [Incumbent — PushPress MOC](#)